

to include an external library for use in the program. This library will help us establish a connection and send/receive data over the internet.

Note. This library should come preinstalled with your IDE. If for some reason you run into errors, please download this library from the official GitHub repository of the Arduino Project.

```
#include <Ethernet.h>
```

In the next step, we will be defining some constants and variables required for this program. First, we need the MAC address. This is often printed on a sticker on the Ethernet shield. Next, we need to define a static IP address. Make sure that the IP address that you are adding isn't being used by any other user. Lastly, we shall define the *EthernetClient* variable.

```
byte mac[] = { 0xDE, 0xAD, 0xBE, 0xEF, 0xFE, 0xED };
IPAddress staticIP(10, 0, 0, 20);

EthernetClient client;
```

We can now write the method for connecting to the internet. There are two steps here. First, we try to get an IP via the DHCP (dynamic host configuration protocol), i.e., try to fetch your dynamic IP. If that step fails, then we fall back to the *staticIP* that we have defined above.

```
void connectToInternet()
{
    // Step 1 - Try connecting
    with DHCP
    If (Ethernet.begin(mac) == 0)
    {
        Serial.print("[ERROR]
Failed to connect via DHCP");
        Ethernet.begin(mac,
staticIP); // Connect via static IP
defined earlier
    }
    // Add a delay for initialization
    delay(1000);

    Serial.println("[INFO] Connection
Successful");
    Serial.print("");
```

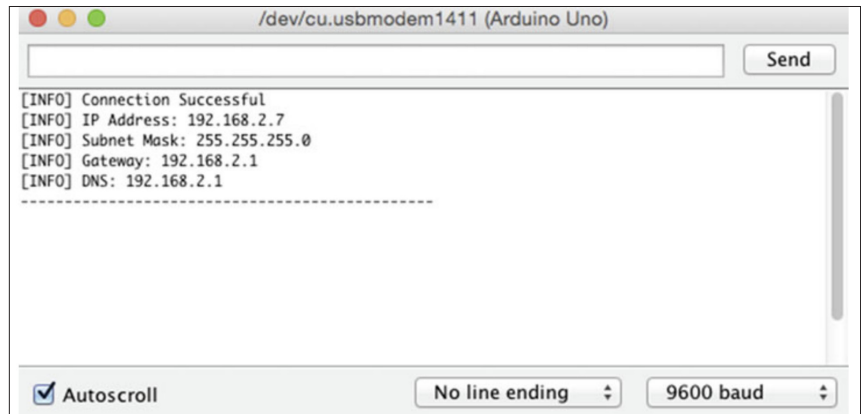


Fig. 3: Successful connection to the internet via Ethernet

```
printConnectionInformation(); // Custom
method
Serial.print
ln("-----");
Serial.println("");
}
```

As you can see in the code above, we used a custom method *printConnectionInformation()* for displaying the connection information. So let us go ahead and write the code for that.

```
void printConnectionInformation()
{
    // Print IP Address
    Serial.print("[INFO] IP Address: ");
    Serial.println(Ethernet.localIP());

    // Print Subnet Mask
    Serial.print("[INFO] Subnet Mask: ");
    Serial.println(Ethernet.
subnetMask());

    // Print Gateway
    Serial.print("[INFO] Gateway: ");
    Serial.println(Ethernet.
gatewayIP());

    // Print DNS
    Serial.print("[INFO] DNS: ");
    Serial.println(Ethernet.
dnsServerIP());
}
```

Finally, we shall write the standard functions for the program, i.e., *setup()* and *loop()*.

```
void setup()
{
    Serial.begin(9600);
```

```
// Connect to the internet
connectToInternet();
}

void loop()
{
    // Nothing much to do here.
}
```

If everything checks out, you should be seeing an output similar to Fig. 3 on the serial monitor window.

Connecting via Wi-Fi

The second option is to connect your Arduino to the internet wirelessly via Wi-Fi. If, like most people, you own an Arduino Uno or any other Arduino board that does not have a built-in Wi-Fi capability, you will have to buy a wireless shield separately, like the Ethernet shield.

If you own an Arduino Yún or any other Arduino board with built-in wireless capability, you can skip the 'Hardware requirements' and the description on the next page and get started with the code right away.

Hardware requirements

You will need an Arduino Uno and a wireless shield. Here's how to connect them:

1. Connect the wireless shield on top of the Arduino Uno.
2. Connect the Arduino Uno to your computer via the USB port.